Deep Dive 04 | Nearby Search and Map

Follow geolocation, Geoapify Places, normalization, caching and Leaflet marker rendering.

React/Vite Edition | Repository-grounded study material | Snapshot: 2026-10-24

1. Browser location and fallback

The nearby page calls navigator.geolocation. If unavailable or permission is denied, it uses New Delhi coordinates and sets a visible location notice. The request asks for high accuracy with an eight-second browser timeout. A better user experience would include a manual location picker and explicit fallback state.

2. Request path

The React page fetches /api/nearby?lat=...&lon=...&radius=5000. The Express route converts query parameters to numbers, constrains radius to 500-5000 meters and validates latitude/longitude. It checks the backend Geoapify key, obtains a coordinate/radius cache key, returns fresh data or joins an in-flight request, then calls the Places API.

3. Provider query and categories

The route uses the structured Places POST endpoint with an x-api-key header. Its category set includes hospitals, clinics, police, fire stations, shelters, assembly points and disaster help points. It applies a circle filter and proximity bias, requests up to 500 results, and expects GeoJSON features. The key must remain in backend environment settings.

4. Normalization, distance and map

- toNearbyElement reads a point coordinate and provider category array, then maps each match into the app categories hospital, police, fire_station or shelter.
- The frontend recognizes normalized tags, calculates Haversine distance from the selected location and sorts ascending.
- Filter chips select all places or one category; list entries include distance and directions link.
- NearbyMap dynamically imports Leaflet, creates an OSM tile layer, shows the user location and draws colored markers for returned places.

5. Cache and provider failure

The backend caches results in a process-local Map for five minutes and keeps them eligible for stale service for up to 24 hours. Identical concurrent requests share one pending Promise. Cache keys round coordinates to three decimal places, so nearby requests can share cached results. Multi-instance deployments do not share this cache; it is cleared by process restart.

Current operational caveat: the app has displayed a Geoapify key rejection. Before describing this as working, confirm the deployed backend has an actual Places-enabled project key, valid restrictions/quota and the latest request code. Do not paste keys into the frontend or study material.

6. Known edge cases and next steps

- A valid provider response with no mapped categories can look like a successful empty search.
- Clinics are mapped into the hospital category; explain that this is a product classification choice.
- Shelter/assembly-point data may be sparse; no search result is not proof that no shelter exists.
- The current browser geolocation fallback should be labeled so users do not assume it is their actual location.
- Add provider freshness, status metrics, manual location input and tests for category normalization.

7. Exercise: trace one marker

Choose one sample feature. Record provider category, geometry coordinate order [longitude, latitude], normalized tags, frontend category, distance calculation, marker color and directions URL. This catches common longitude/latitude swaps and category mismatches.